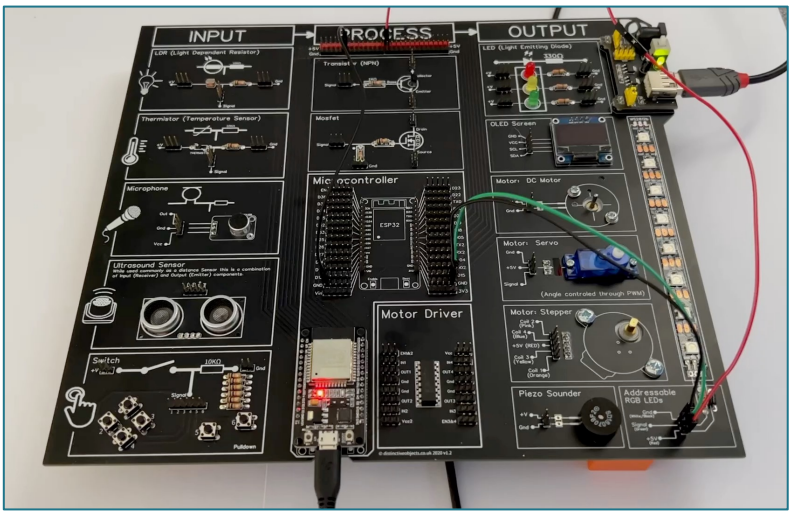
Addressable LEDs

Distinctive Objects



Let's make something flashy!

In this guide we're going to be working with addressable LEDs and we can certainly get them to flash! Not only that, we can also change their colours and control lots of LED with only one microcontroller pin.

What's so great about these addressable LEDs?

We looked, in an earlier guide, at how we can light LEDs. It was quite simple, one pin per LED and a little programme and we were lit up light a Christmas Tree! (Well, a traffic light anyway). This method, while really easy, has a couple of drawbacks:

1. If we are looking at lighting lots of LEDs then that is a lot of microcontroller pins being used.
2. We are pretty limited on colour – Our Red, amber and green LEDs are fixed and can't be changed
3. If we start driving lots of LEDs from just the ESP32s output pins the microcontroller is not going to be happy!

Addressable LEDs on the other hand allow you to control hundreds of LEDs (power source allowing) all controlled by a single Microcontroller pin!

Addressable LEDs?

Let's consider your address at home, that tells someone, like the person delivering your letters, that it should go to you and no one else. The same is true for these LEDs, each has its own specific address that we can send a signal to dependent on where they are along the whole strip. This means if we have a strip of 60 LEDs we can choose light any on their own or have all the LEDs on the strip on.

Also, as each LED 'chip' is made up from 3 LEDs (red, green and blue) we can also get a fantastic range of colours by mixing the colours together at different intensities and brightness (Think of the paints you mixed back in art class and that gives you an idea of what is going on)

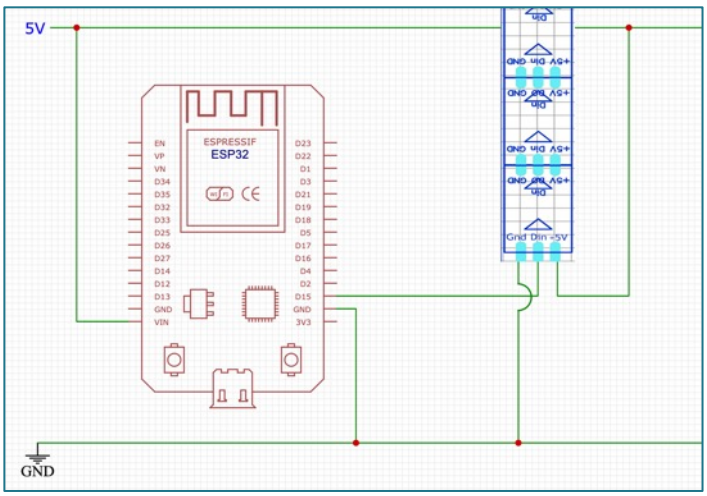
Let’s get started!

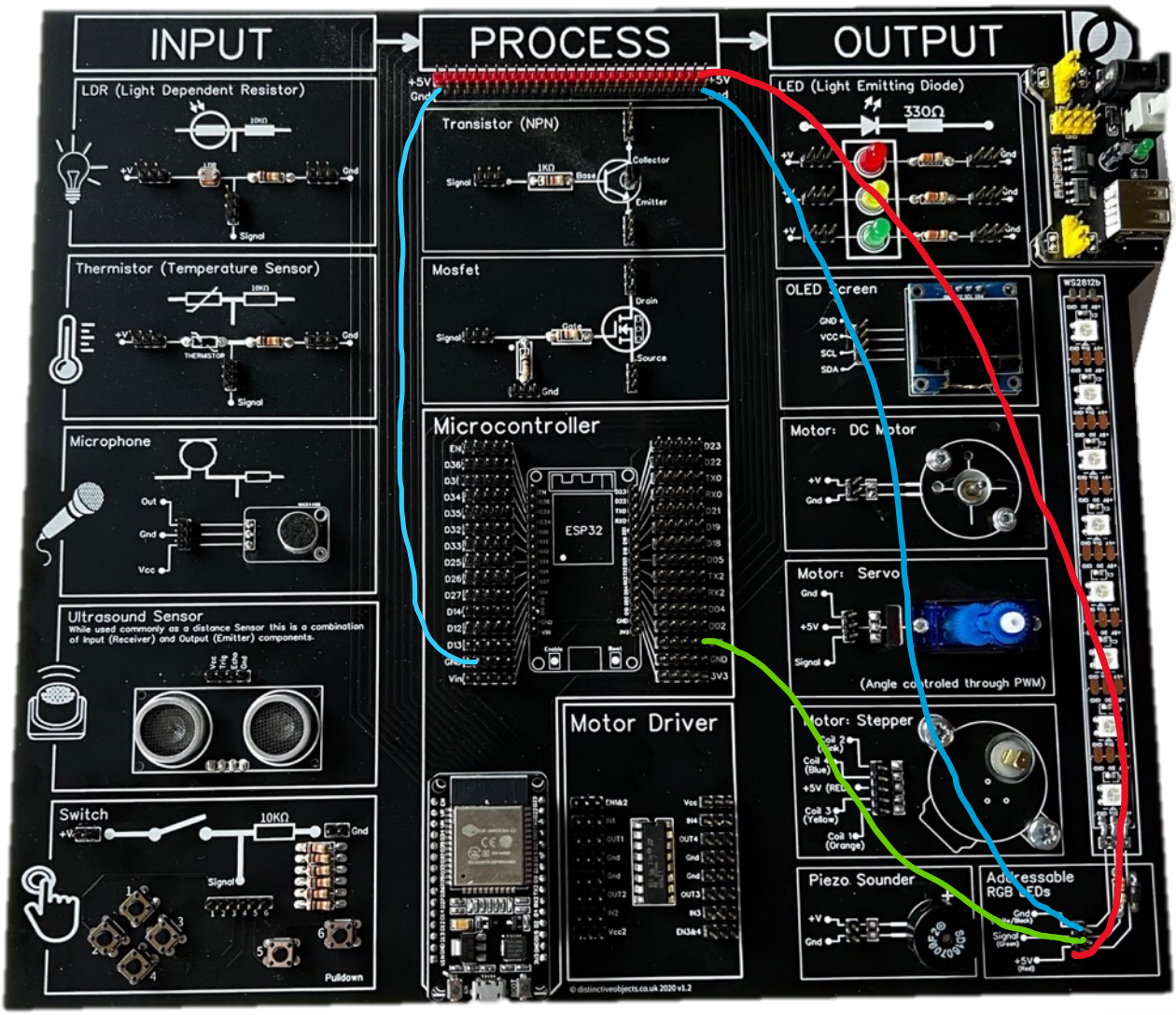
Make the connections.
The connections to the addressable LED strip are quite simple:

5V - Addressable LEDs need 5V to work well
Connect the 5V external supply for this.

Gnd – This connection is to the external power supply’s ground (Gnd) but also must be connected to the Gnd on the ESP32. This method is called *common ground and is necessary whenever we are using a power source which is not part of the ESP32.*

Din *– the Pin we are going to use to control this is D15*





Your board should look like the above.

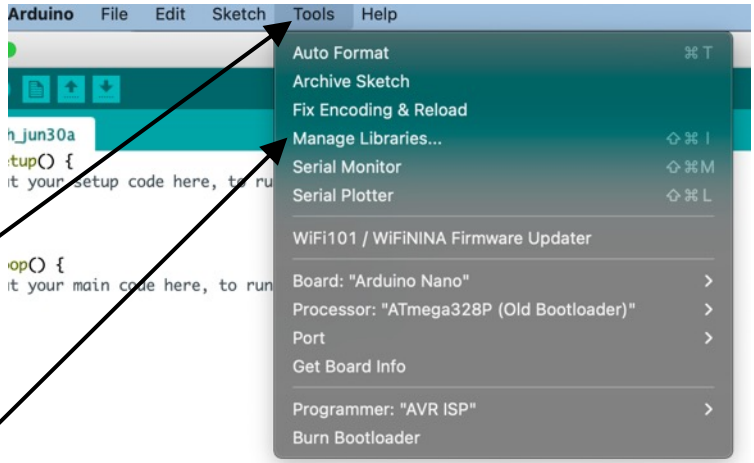
Remember it doesn’t matter what colour wires you use as long as the connections you make are right.

Programming out LEDs

Neopixel/ Fast LED Library

To control our LEDs we are going to install a library to help us. A library is basically a block of coding that someone has done for us and takes the complexity out of a lot of tasks.

For controlling addressable LEDs there are two main choices of libraries that we can use: Adafruit's Neopixel library and Fast LED.

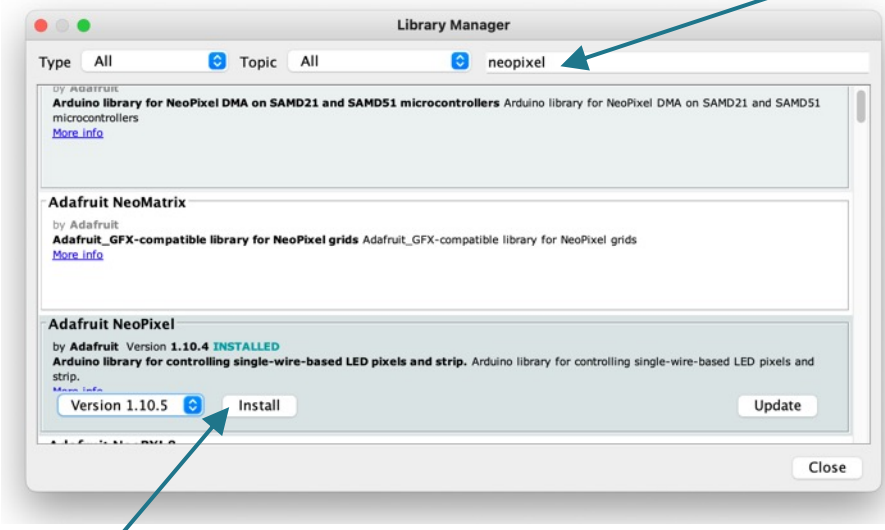


Installing Libraries

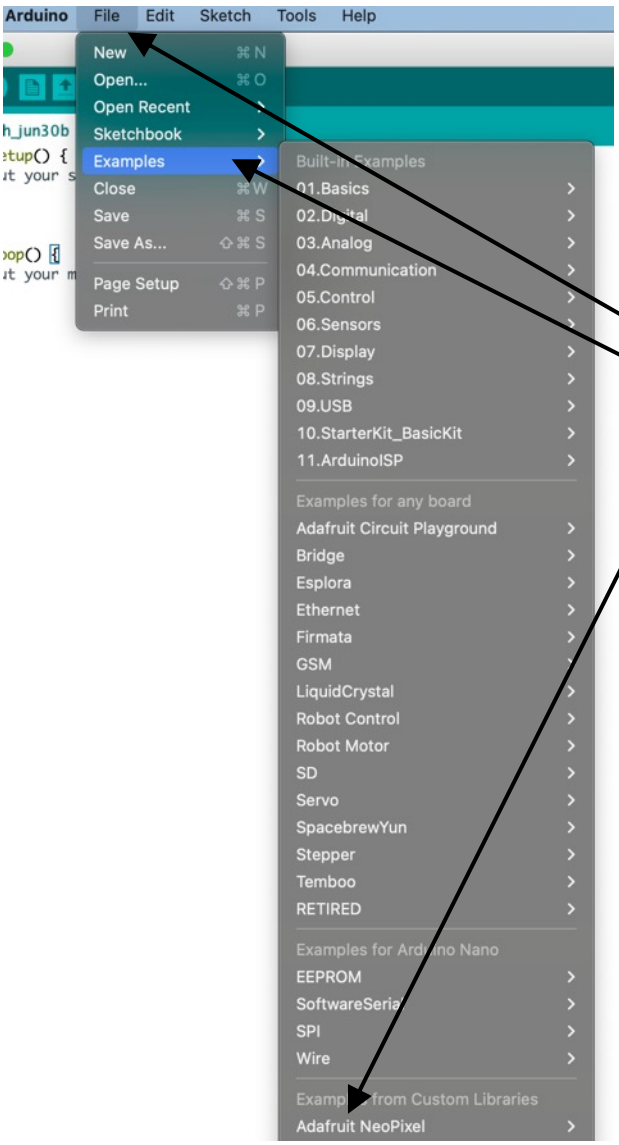
First part of installing this is to go to the library manager. This can be found in the **Tools** link at the top of the Arduino IDE.

Next click on **Manage Libraries**

Once the library manager has opening you will see a search bar at the top. Type in ***'neopixel'***.



After a few seconds the library list will update and you will see the Adafruit Neopixel library, click **install**.



Let there be light!

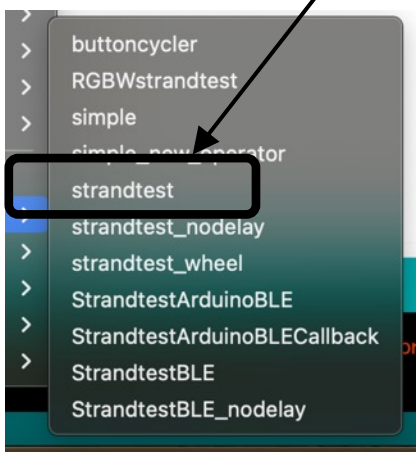
You should be ready, lets see if it works.

We're going to use one of the examples built into the Neopixel library to test our circuit.

- Goto the ***Files*** menu
- choose ***examples*** on the Arduino IDE.

At the bottom of the list that opens will be the Neopixel library (If not, go back to the step about installing libraries and give that another go.

A menu will pop up, choose ***strandtest***



Example Code

At this point the code for performing a strand test will open. This is just a generic piece of code that's designed to work with lots of types of addressable LEDs.

Fortunately, the code we have to change is easy and outlined on the next page.

```
strandtest
// A basic everyday NeoPixel strip test program.

// NEOPIXEL BEST PRACTICES for most reliable operation:
// - Add 1000 uF CAPACITOR between NeoPixel strip's + and - connections.
// - MINIMIZE WIRING LENGTH between microcontroller board and first pixel.
// - NeoPixel strip's DATA-IN should pass through a 300-500 OHM RESISTOR.
// - AVOID connecting NeoPixels on a LIVE CIRCUIT. If you must, ALWAYS
//   connect GROUND (-) first, then +, then data.
// - When using a 3.3V microcontroller with a 5V-powered NeoPixel strip,
//   a LOGIC-LEVEL CONVERTER on the data line is STRONGLY RECOMMENDED.
// (Skipping these may work OK on your workbench but can fail in the field)

#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
 #include <avr/power.h> // Required for 16 MHz Adafruit Trinket
#endif

// Which pin on the Arduino is connected to the NeoPixels?
// On a Trinket or Gemma we suggest changing this to 1:
#define LED_PIN    6

// How many NeoPixels are attached to the Arduino?
#define LED_COUNT 60

// Declare our NeoPixel strip object:
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
```

We're almost there!

Customising the code

In this we need to change two aspects:

LED_PIN & LED_COUNT

LED_PIN: This IS the pin that the Din of the Addressable LEDs is connected to on our ESP32. In our case it was Pin 15.

Change the line to :

#define LED_PIN 15

LED_COUNT: This is asking how many LEDs are connected. On our board we have 8 LEDs.

Change the line to :

#define LED_COUNT 8

Original Code

strandtest

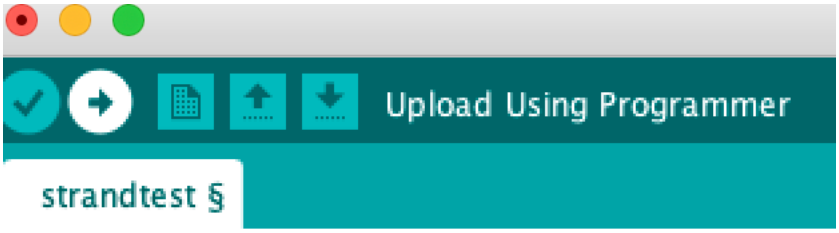
```
// A basic everyday NeoPixel strip test program.

// NEOPIXEL BEST PRACTICES for most reliable operation:
// - Add 1000 uF CAPACITOR between NeoPixel strip's + and - connections.
// - MINIMIZE WIRING LENGTH between microcontroller board and first pixel.
// - NeoPixel strip's DATA-IN should pass through a 300-500 OHM RESISTOR.
// - AVOID connecting NeoPixels on a LIVE CIRCUIT. If you must, ALWAYS
//   connect GROUND (-) first, then +, then data.
// - When using a 3.3V microcontroller with a 5V-powered NeoPixel strip,
//   a LOGIC-LEVEL CONVERTER on the data line is STRONGLY RECOMMENDED.
// (Skipping these may work OK on your workbench but can fail in the field)

#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
 #include <avr/power.h> // Required for 16 MHz Adafruit Trinket
#endif

// Which pin on the Arduino is connected to the NeoPixels?
// On a Trinket or Gemma we suggest changing this to 1:
#define LED_PIN    6

// How many NeoPixels are attached to the Arduino?
#define LED_COUNT 60

// Declare our NeoPixel strip object:
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
```

Altered Code

```
#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
 #include <avr/power.h> // Required for 16 MHz Adafruit Trinket
#endif

// Which pin on the Arduino is connected to the NeoPixels?
// On a Trinket or Gemma we suggest changing this to 1:
#define LED_PIN    15

// How many NeoPixels are attached to the Arduino?
#define LED_COUNT 8

// Declare our NeoPixel strip object:
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
// Argument 1 = Number of pixels in NeoPixel strip
// Argument 2 = Arduino pin number (most are valid)
```

The code should look like the screenshot on the right.

<u>Upload the programme</u>



Your LEDs should light up, if not, press the En button to restart the ESP32.

If it's still not working, check your connections and make sure the Gnd on the ESP32 is connected to Gnd on the power supply.